Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

Autómatas y Lenguajes Formales

ER-AFD

Convertidor de expresiones regulares a autómatas finitos deterministas

REPORTE TÉCNICO

Catedrático: René Ornélyz

Estudiante: Josue Garcia

Guatemala, octubre de 2026

Contenido

1. Introducción
2. Arquitectura del sistema
3. Análisis sintáctico de la expresión regular
4. Representación de los símbolos
5. Conversión ER → AFND: algoritmo de Thompson
6. Conversión AFND → AFD: construcción de subconjuntos
7. Evaluación de cadenas
8. Minimización del AFD (funcionalidad adicional)
9. Validación y métricas de precisión
10. Decisiones de diseño
11. Referencias

1. Introducción

ER-AFD es una herramienta de línea de comandos que recibe una expresión regular (ER) y una cadena de texto, construye el autómata finito equivalente y decide si la cadena pertenece al lenguaje descrito por la ER. La conversión sigue el camino clásico de la teoría de autómatas:

ER → **AFND** (algoritmo de Thompson) → **AFD** (construcción de subconjuntos) → **evaluación** con la matriz de transiciones

El programa muestra la tabla de transiciones del AFND y del AFD, y luego el recorrido paso a paso de la cadena. Termina con `Cadena aceptada` (código de retorno 1) o `Cadena rechazada` (código de retorno 0).

Todo el núcleo está implementado desde cero en Python, solo con la biblioteca estándar: el analizador sintáctico, la construcción de los autómatas y el simulador. No se usa ninguna librería de expresiones regulares (`re` u otras). Una prueba automática revisa todos los archivos del proyecto para garantizarlo.

2. Arquitectura del sistema

El procesamiento es una cadena de transformaciones independientes; cada módulo recibe la estructura del anterior:

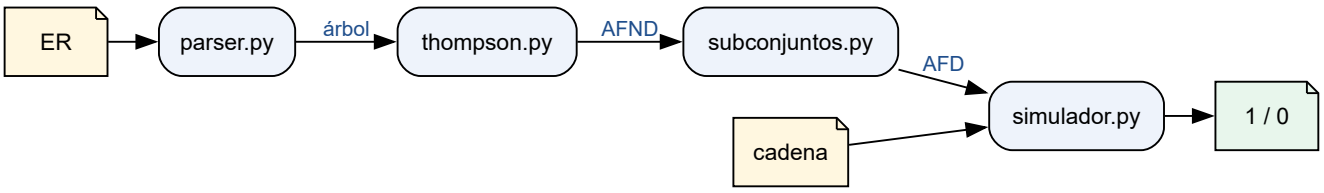


Figura 1. Flujo de datos entre los módulos.

Módulo	Responsabilidad
`simbolos.py`	Conjuntos de caracteres ASCII, clases predefinidas (`\N`, `\A`, `\a`, `\z`) y su representación textual.
`parser.py`	Analizador sintáctico descendente recursivo: convierte el texto de la ER en un árbol sintáctico y reporta errores con su posición.
`arbol.py`	Nodos del árbol: `Simbolo`, `Epsilon`, `Concatenacion`, `Union`, `Cerradura`.
`thompson.py`	Construcción de Thompson: árbol → AFND con transiciones ε.
`subconjuntos.py`	Construcción de subconjuntos: AFND → AFD, con cerradura-ε, mover y partición del alfabeto en clases disjuntas.
`simulador.py`	Recorrido de la cadena con la matriz de transiciones del AFD.
`formato.py`	Impresión de tablas y recorrido con el formato `estado|símbolo|destino`.

`minimizacion.py`, `dot.py`	Funcionalidades adicionales: AFD mínimo (Moore) y exportación de diagramas a Graphviz.
`cli.py`	Interfaz de línea de comandos `er-afd [opciones] <ER> <cadena>` y códigos de retorno.

3. Análisis sintáctico de la expresión regular

3.1 Gramática

La ER se analiza con un **analizador descendente recursivo** escrito a mano. Cada regla de la gramática es un método, y el orden de las reglas define la precedencia de los operadores, de menor a mayor: unión, concatenación (implícita) y cerradura de Kleene.

```
union         := concatenacion ( '|' concatenacion )*
concatenacion := repeticion repeticion*                (concatenación implícita)
repeticion    := atomo '*'*
atomo         := '(' union ')' | conjunto | '.' | 'ε' | escape | literal
conjunto      := '[' '^'? elemento+ ']'
elemento      := caracter ( '-' caracter )? | clase_escape
```

Una regla de menor precedencia llama a las de mayor precedencia, así que `ab|c*` se agrupa como `(ab)|(c*)`. La unión y la concatenación se construyen de forma iterativa y asociativa por la izquierda: `a|b|c` = `(a|b)|c`.

3.2 Árbol sintáctico

El resultado del análisis es un árbol cuyas hojas son símbolos (o ε) y cuyos nodos internos son operadores. Para la expresión del ejemplo guía de este reporte, `(a|b)*abb`:

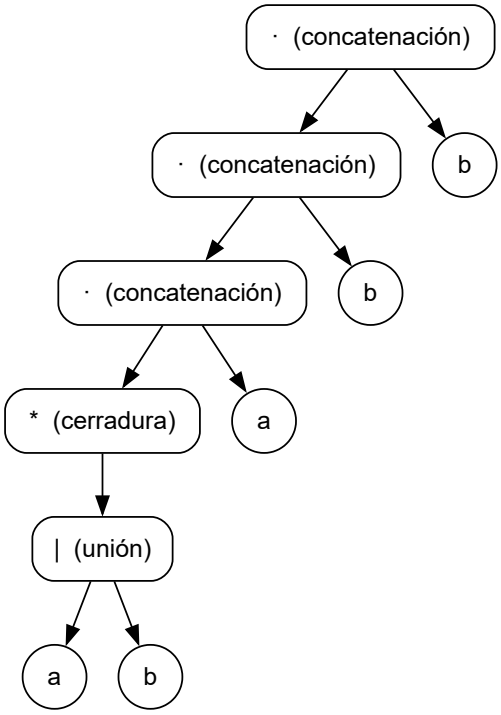


Figura 2. Árbol sintáctico de `(a|b)*abb`.

3.3 Elementos soportados

Sintaxis	Interpretación en el analizador
`ab`, `a|b`, `a*`, `(r)`	Concatenación, unión, cerradura de Kleene y agrupación.
`.`	Cualquier símbolo ASCII (códigos 0 a 127).
`ε`	Cadena vacía: genera una hoja `Epsilon`.
`[abc]`, `[a-z]`, `[^abc]`	Conjunto, rango y conjunto negado (complemento respecto del ASCII). Dentro de un conjunto se admiten escapes y clases: `[\N_]`. Un `-` al inicio o al final es literal.
`\x`	El carácter `x` sin significado especial: `\*`, `|`, `\\`, `\ `, `\p`.
`\N`, `\A`, `\a`, `\z`	Dígito, mayúscula, minúscula y letra.
`\hHH`	Carácter por su código ASCII hexadecimal; exige exactamente dos dígitos (`\h41` = `A`).

3.4 Manejo de errores

Una ER mal formada produce un mensaje que explica el problema y señala la posición, y el programa termina con código 2. Se detectan, entre otros: paréntesis vacíos o sin cerrar, alternativas vacías (`a|`, `|a`), `*` sin operando, conjuntos vacíos o sin cerrar, rangos invertidos (`[z-a]`), `\h` incompleto o fuera del ASCII, caracteres no ASCII y `ε` dentro de un conjunto.

```
er-afd: error de sintaxis en la expresión regular: rango invertido 'z-a': el inicio es
mayor que el final (posición 2)
  [z-a]
    ^
```

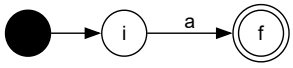
4. Representación de los símbolos

Cada hoja del árbol se representa como el **conjunto de caracteres ASCII que acepta**: la letra `a` es {97}, `\N` es {48,...,57} y `.` es {0,...,127}. Así los literales, los conjuntos, los rangos y las clases se tratan de la misma forma en todos los algoritmos. Dos símbolos con el mismo conjunto, como `\N` y `[0-9]`, son el mismo símbolo del alfabeto.

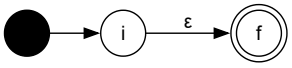
El alfabeto Σ del AFND contiene los símbolos en el orden en que aparecen en la ER. Si `ε` aparece, ocupa su lugar en ese orden; si no, se agrega al final, como pide el enunciado. Los caracteres que son operadores se muestran escapados (`\|`, `\.`, `\*`), y el espacio y los caracteres de control se muestran como `\hXX`. Así ningún símbolo se confunde con el separador `|` de las tablas.

5. Conversión ER → AFND: algoritmo de Thompson

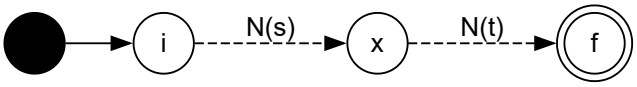
El algoritmo de Thompson (1968) construye el AFND de forma recursiva sobre el árbol sintáctico. Cada subexpresión *r* produce un fragmento N(*r*) con un único estado inicial *i*, sin transiciones de entrada, y un único estado final *f*, sin transiciones de salida. Los fragmentos se combinan con estas reglas:



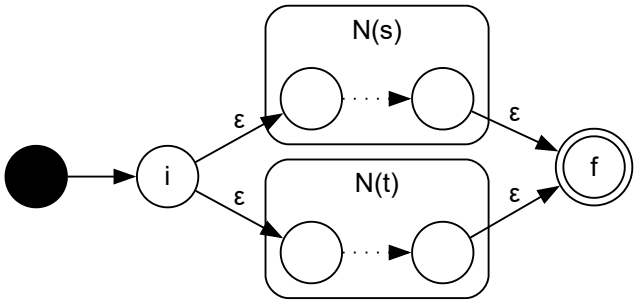
Símbolo *a*



Cadena vacía ε



Concatenación *st*: el final de N(*s*) se fusiona con el inicio de N(*t*) (estado *x*)



Unión *s*|*t*

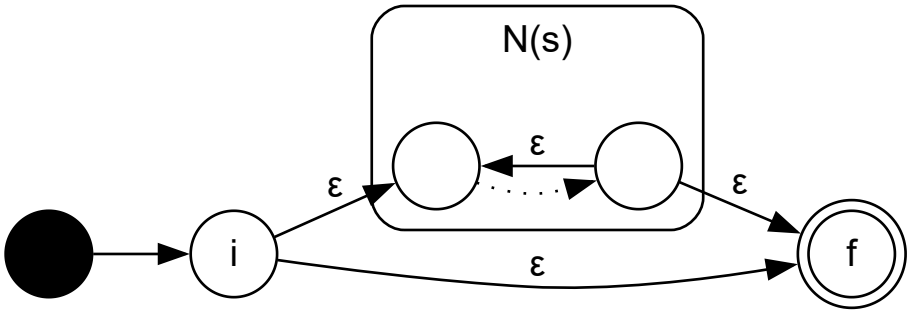


Figura 3. Cerradura *s**: permite saltar N(*s*) (cero repeticiones) o regresar a su inicio (más repeticiones).

5.1 Implementación

La función `_construir` recorre el árbol y devuelve, para cada nodo, su estado inicial, su estado final y la lista ordenada de sus estados. En la concatenación, el estado final de N(*s*) hereda las transiciones del inicio de N(*t*), y ese inicio se descarta. Es seguro porque, por construcción, el final de un fragmento no tiene transiciones de salida y su inicio no tiene transiciones de entrada.

Los estados se numeran en **orden estructural**: el inicio de cada construcción, luego los estados de sus partes y al final su estado final. Así `q0` es siempre el inicial y el último estado es el único de aceptación. Para `(a|b)*abb` se obtiene exactamente la numeración 0–10 de la figura 3.34 del libro de Aho et al.

5.2 Ejemplo: `(a|b)*abb`

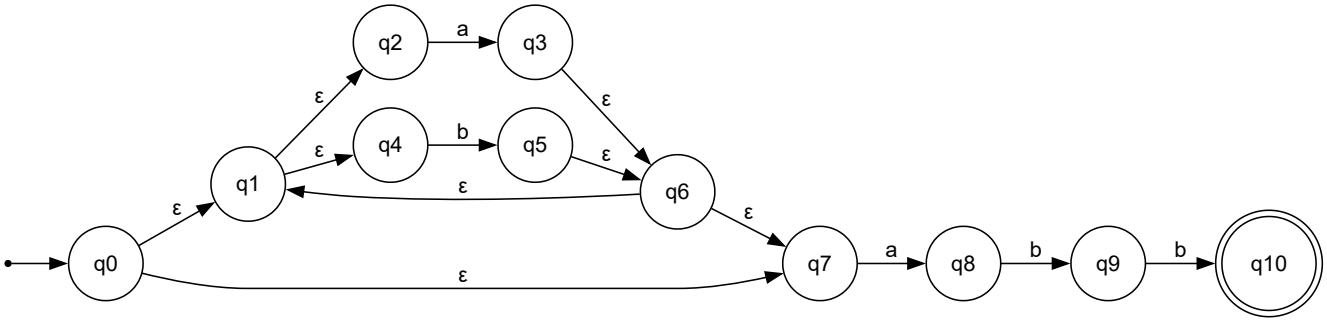


Figura 4. AFND de Thompson para `(a|b)*abb` (11 estados).

```
AFND
Q={q0,q1,q2,q3,q4,q5,q6,q7,q8,q9,q10}
Σ={a,b,ε}
F={q10}
q0|ε|{q1,q7}     q3|ε|{q6}     q6|ε|{q1,q7}     q9|b|{q10}
q1|ε|{q2,q4}     q4|b|{q5}     q7|a|{q8}
q2|a|{q3}        q5|ε|{q6}     q8|b|{q9}
```

Tamaño: cada símbolo aporta 2 estados, cada unión y cada cerradura 2 más, y cada concatenación resta 1 por la fusión. Por eso el AFND tiene a lo sumo 2·*m* estados para una ER con *m* símbolos y operadores. Cada estado tiene como máximo dos transiciones de salida, y la construcción es lineal en el tamaño de la ER.

6. Conversión AFND → AFD: construcción de subconjuntos

Cada estado del AFD representa el **conjunto de estados** en los que puede estar el AFND después de leer la misma entrada. Se usan dos operaciones:

- **cerradura-ε(T)**: estados alcanzables desde T usando solo transiciones ε. Se calcula con una pila.
- **mover(T, c)**: estados alcanzables desde T con una transición sobre el símbolo c.

```
Q0 := cerradura-ε({q0});  Destados := [Q0]
mientras exista un estado T sin marcar en Destados:
    marcar T
    para cada símbolo c de Σ (en orden):
        U := cerradura-ε(mover(T, c))
        si U ≠ ∅:
            si U no está en Destados: agregar U sin marcar
            Dtran[T, c] := U
F := { T ∈ Destados | T contiene el estado final del AFND }
```

Los estados se numeran Q0, Q1, … en el orden en que se descubren. Las transiciones al conjunto vacío no se agregan, así que el AFD resultante es **parcial**: si un carácter no tiene transición, la cadena se rechaza en ese paso.

6.1 Ejemplo: `(a|b)*abb`

Q0 = cerradura-ε({q0}) = {q0, q1, q2, q4, q7}. Con la entrada `a`, mover(Q0, a) = {q3, q8}; su cerradura es {q1, q2, q3, q4, q6, q7, q8}, un estado nuevo: Q1. Con `b`, mover(Q0, b) = {q5}, cuya cerradura es Q2. El proceso continúa hasta que no aparecen conjuntos nuevos:

Estado AFD	Subconjunto del AFND	a	b	¿Final?
Q0	{q0, q1, q2, q4, q7}	Q1	Q2	no
Q1	{q1, q2, q3, q4, q6, q7, q8}	Q1	Q3	no
Q2	{q1, q2, q4, q5, q6, q7}	Q1	Q2	no
Q3	{q1, q2, q4, q5, q6, q7, q9}	Q1	Q4	no
Q4	{q1, q2, q4, q5, q6, q7, q10}	Q1	Q2	sí (contiene q10)

El resultado coincide con los estados A–E del ejemplo 3.21 de Aho et al. La opción `-v` del programa imprime esta composición.

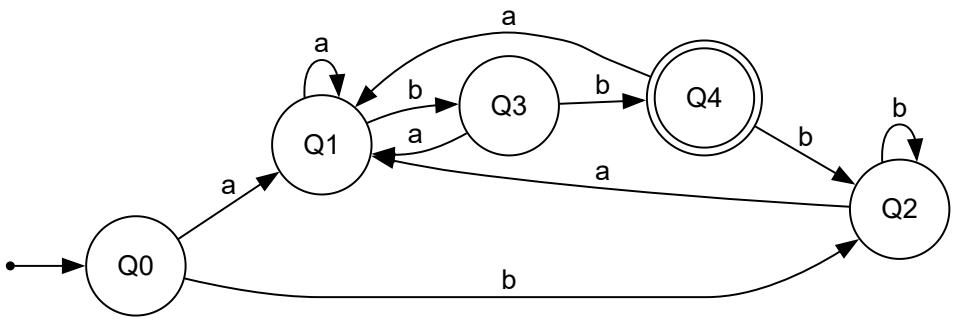


Figura 5. AFD obtenido por construcción de subconjuntos (Q4 es de aceptación).

6.2 Símbolos que se solapan: partición en clases disjuntas

Los símbolos de una ER pueden compartir caracteres. En `a.b`, la letra `a` pertenece al símbolo `a` y también al símbolo `.`. Si `a` y `.` se tomaran como columnas independientes, desde un mismo estado la entrada `a` tendría dos transiciones posibles y el autómata no sería determinista.

Para evitarlo, antes de la construcción el alfabeto se divide en **clases disjuntas**: dos caracteres van a la misma clase si pertenecen exactamente a los mismos símbolos de la ER. Cada carácter de la entrada cae así en una sola clase, y el AFD resultante es determinista siempre. Las clases se nombran de forma legible:

- si una clase coincide con un símbolo de la ER, lleva su nombre (`a`, `[a-z]`, `\N`);
- la clase de los caracteres que solo cubre `.` se llama `.` y significa "cualquier otro símbolo";
- cualquier otra clase se escribe como conjunto; por ejemplo, en `[a-z]*a` aparece `[b-z]`.

Si los símbolos no se solapan, como en `(a|b)*abb`, las clases coinciden con Σ y la tabla es la del enunciado. Para `a.b`, Σ del AFD es {a, ., b}:

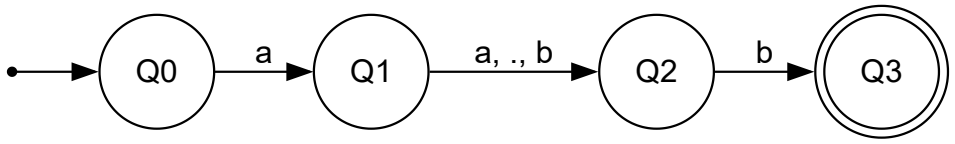


Figura 6. AFD de `a.b`: desde Q1 cualquier carácter lleva a Q2, separado en las clases a, b y "cualquier otro".

7. Evaluación de cadenas

El AFD se almacena como una **matriz de transiciones**: para cada estado, un diccionario que asocia el índice de la clase de símbolo con el estado destino. Un arreglo de 128 posiciones indica a qué clase pertenece cada código ASCII. Cada paso de la simulación cuesta O(1), y la evaluación completa O(|w|).

```
estado := Q0
para cada carácter c de la cadena:
    destino := Dtran[estado, clase(c)]
    imprimir  estado|c|destino            ({} si no existe la transición)
    si no existe destino: rechazar
    estado := destino
aceptar si estado ∈ F; si no, rechazar
```

Recorrido de `aabb` en el AFD de la figura 5:

```
Q0|a|Q1
Q1|a|Q1
Q1|b|Q3
Q3|b|Q4
Cadena aceptada
```

El valor de retorno del programa es **1** si la cadena se acepta y **0** si se rechaza, como pide el enunciado. Una ER inválida devuelve **2**. Si la cadena contiene un carácter sin transición, incluidos los caracteres fuera del ASCII, el recorrido muestra `Qi|c|{}` y la cadena se rechaza. Se usa `{}`, la notación del conjunto vacío del propio formato, en lugar del carácter ∅, porque ∅ no existe en las fuentes de muchas terminales.

8. Minimización del AFD (funcionalidad adicional)

Con la opción `-m`, el programa calcula también el AFD mínimo con el algoritmo de Moore (refinamiento de particiones):

1. Se completa el AFD con un estado sumidero implícito.
2. Partición inicial: {estados de aceptación, estados de no aceptación}.
3. Dos estados de un mismo bloque se separan si, con algún símbolo, llevan a bloques distintos. Se repite hasta que la partición no cambia.
4. Cada bloque es un estado del AFD mínimo, con nombre `Mi`; el bloque del sumidero se descarta.

En el ejemplo, Q0 y Q2 son equivalentes, así que el AFD mínimo tiene 4 estados, como en el libro:

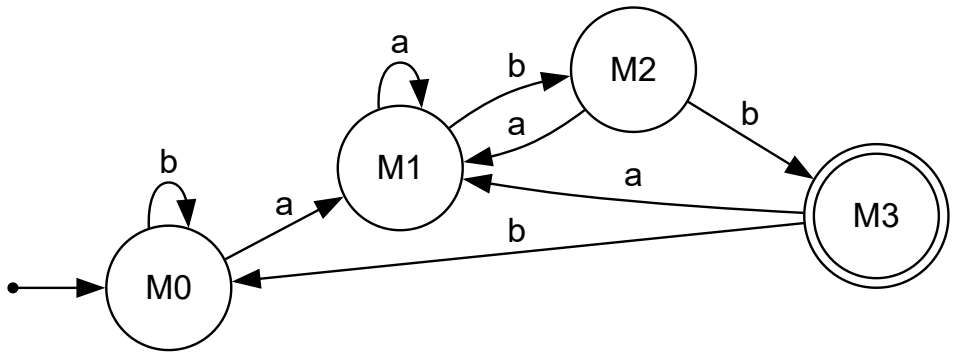


Figura 7. AFD mínimo de `(a|b)*abb`: M0 = {Q0, Q2}.

9. Validación y métricas de precisión

La corrección se verifica con tres mecanismos independientes, automatizados con `pytest` (358 pruebas) y con `make metrics`:

1. **Oráculo independiente.** Un emparejador recursivo que decide la pertenencia recorriendo directamente el árbol sintáctico, sin construir autómatas. Para cada subexpresión calcula las posiciones de la cadena en las que puede terminar. Como no comparte código con Thompson ni con los subconjuntos, un error en los autómatas se detecta como un desacuerdo.
2. **Corpus de detecciones reales.** 23 patrones de uso práctico: identificadores, números, correos, teléfonos y placas de Guatemala, carné UMG, DPI, fechas, horas, direcciones IPv4, URL, comentarios de C, entre otros. Suman 207 cadenas etiquetadas a mano.
3. **Pruebas aleatorias.** 500 ER generadas al azar con todos los operadores, cada una evaluada con 40 cadenas. Se compara el AFD, el AFND, el AFD mínimo y el oráculo.

Métrica (corpus de detecciones reales)	Resultado
Casos evaluados	207 (94 positivos, 113 negativos)
Verdaderos positivos / verdaderos negativos	94 / 113
Falsos positivos / falsos negativos	0 / 0
Exactitud, precisión, exhaustividad, F1	100 %
Verificación aleatoria (20 000 cadenas)	100 % de coincidencia

9.1 El autómata más complejo: dirección IPv4

La ER que valida una dirección IPv4 con octetos de 0 a 255 repite cuatro veces el patrón de un octeto:

```
(\N|[1-9]\N|1\N\N|2[0-4]\N|25[0-5])\.(…)\.(…)\.(…)
```

Produce un AFND de 100 estados, un AFD de 48 y un AFD mínimo de 24. Es un buen ejemplo de clases disjuntas: `\N`, `[1-9]`, `[0-4]`, `[0-5]`, `1`, `2` y `5` se solapan, y el alfabeto queda dividido en 7 clases, que el programa muestra como Σ = {0, [34], [6-9], 1, 2, 5, \.}. El AFD mínimo de un solo octeto muestra cómo se distinguen los casos 0–9, 10–99, 100–199, 200–249 y 250–255:

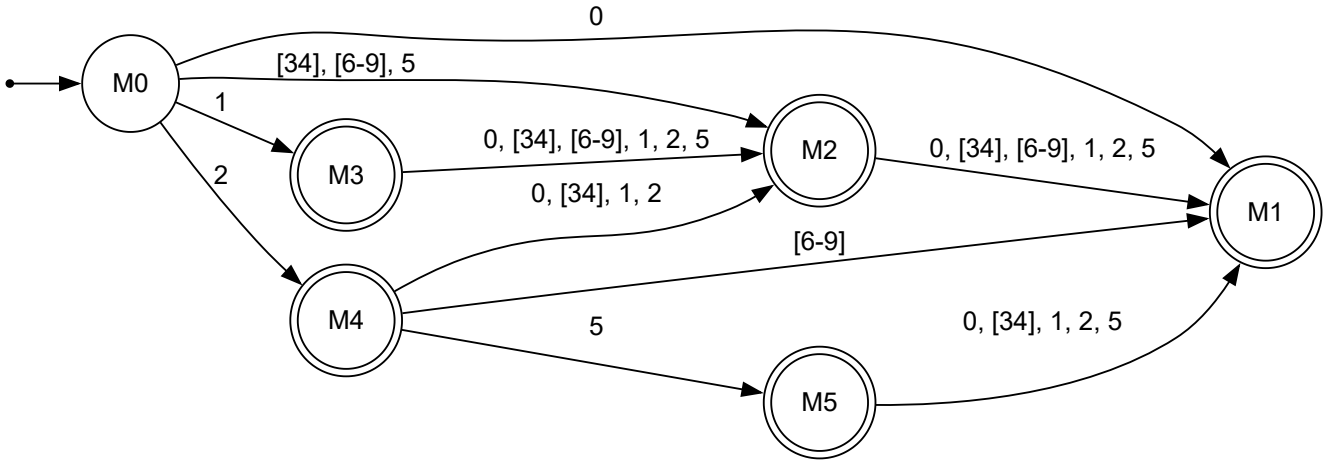


Figura 8. AFD mínimo de un octeto (0–255).

10. Decisiones de diseño

Decisión	Justificación
Símbolos como conjuntos de caracteres y partición en clases disjuntas	Permite usar `.`, conjuntos y clases sin perder el determinismo del AFD (sección 6.2). Cuando no hay solapamiento, Σ coincide con la del enunciado.
AFD parcial, sin estado sumidero en la tabla	El ejemplo del enunciado lista Q sin estado sumidero. Cuando falta una transición, el recorrido muestra `{}` y rechaza.
Concatenación con fusión de estados y numeración estructural	Es la construcción del libro de referencia: produce menos estados y la misma numeración que el ejemplo clásico, lo que facilita verificar el resultado.
Analizador descendente recursivo propio	Muestra el dominio de la gramática, da mensajes de error precisos con posición y no añade dependencias.
`+`, `?`, `^`, `-` literales fuera de un conjunto	El enunciado no los define como operadores. Interpretarlos como operadores cambiaría el significado de ER válidas.
Códigos de retorno 1, 0 y 2	1 y 0 son los que exige el enunciado; 2 distingue los errores de uso o de sintaxis. El `Makefile` trata el 1 como éxito en `make run`.
Sin dependencias de ejecución; ejecutable `zipapp`	`make build` genera un único archivo ejecutable `bin/er-afd` que funciona con cualquier Python 3.8 o superior, sin root.
Validación sin librerías de regex	El oráculo de las pruebas también es propio, para cumplir la restricción del enunciado sin excepciones.

11. Referencias

1. A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman. *Compiladores: principios, técnicas y herramientas*, 2.ª ed. Pearson Educación, 2008. Secciones 3.7.1 (conversión de un AFN en AFD), 3.7.4 (construcción de un AFN a partir de una expresión regular) y 3.9.6 (minimización del número de estados de un AFD).
2. K. Thompson. "Programming Techniques: Regular expression search algorithm". *Communications of the ACM*, 11(6), pp. 419–422, 1968.
3. J. E. Hopcroft, R. Motwani y J. D. Ullman. *Introducción a la teoría de autómatas, lenguajes y computación*, 3.ª ed. Pearson, 2007. Capítulos 2 y 4.
4. E. F. Moore. "Gedanken-experiments on sequential machines". *Automata Studies*, Princeton University Press, pp. 129–153, 1956.